



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Développement web back-end

Les variables superglobales

Sommaire

1	Les variables superglobales en PHP	2
1.1	Présentation	2
2	Création et gestion des cookies en PHP	2
2.1	Présentation des cookies	2
2.2	Créer un cookie en PHP	3
2.3	Récupérer la valeur d'un cookie	4
2.4	Modifier ou supprimer un cookie.....	5
3	Définir et utiliser les sessions en PHP	5
3.1	Présentation des sessions en PHP	5
3.2	Démarrer une session en PHP	6
3.3	Définir et récupérer des variables de session	7
3.4	Terminer une session et détruire les variables de session.....	8

1 Les variables superglobales en PHP

1.1 Présentation

Les variables superglobales sont des variables internes au PHP, ce qui signifie que ce sont des variables créées automatiquement par le PHP.

Ces variables vont être accessibles n'importe où dans le script et quel que soit le contexte, qu'il soit local ou global. C'est d'ailleurs la raison pour laquelle on appelle ces variables « superglobales ».

Il existe 9 variables superglobales. Ces variables sont toutes des variables tableaux.

Les variables superglobales PHP sont les suivantes :

- `$GLOBALS`
- `$_SERVER`
- `$_REQUEST`
- `$_GET`
- `$_POST`
- `$_FILES`
- `$_ENV`
- `$_COOKIE`
- `$_SESSION`

Par convention, les superglobales sont toujours écrites en majuscule. De même, mis à part `$GLOBALS` les superglobales commencent par un underscore `_`.

Certaines de ces variables sont plus utilisées que d'autres et nous aurons l'occasion d'en faire usage plus tard.

Documentation sur les [superglobales en PHP](#).

2 Création et gestion des cookies en PHP

2.1 Présentation des cookies

Un cookie est un petit fichier texte qui ne peut contenir qu'une quantité limitée de données.

Les cookies vont être stockés sur les ordinateurs de vos visiteurs. Ainsi, à tout moment, un utilisateur peut lui-même supprimer les cookies de son ordinateur.

De plus, les cookies vont toujours avoir une durée de vie limitée. On pourra définir la date d'expiration d'un cookie.

Généralement, nous allons utiliser les cookies pour faciliter la vie des utilisateurs en préenregistrant des données les concernant comme un nom d'utilisateur par exemple.

Ainsi, dès qu'un utilisateur connu demande à accéder à une page de notre site, les cookies vont également automatiquement être envoyés dans la requête de l'utilisateur. Cela va nous permettre de l'identifier et de lui proposer une page personnalisée.

Les cookies ne sont donc pas dangereux en soi même s'ils continuent d'avoir mauvaise réputation. En revanche, on évitera toujours de stocker des informations sensibles dans les cookies comme des mots de passe par exemple car les cookies sont stockés sur l'ordinateur des visiteurs et nous n'avons donc aucune maîtrise ni aucun moyen de les sécuriser après le stockage.

2.2 Créer un cookie en PHP

Pour créer un cookie en PHP, nous allons utiliser la fonction `setcookie()`.

Pour qu'elle fonctionne, cette fonction doit être appelée avant d'écrire tout code html puisque les cookies doivent être envoyés avant toute autre sortie.

Cette fonction peut accepter jusqu'à sept valeurs en arguments. Cependant, seul la première (le nom du cookie créé) est obligatoire.

La syntaxe de base de `setcookie()` est la suivante :

```
setcookie(name, value, expire, path, domain, secure, httponly).
```

Les paramètres ont la signification suivante :

Paramètre	Signification
name	Le nom du cookie (soumis aux mêmes règles que les noms de variables).
value	La valeur du cookie. Comme le cookie est stocké sur l'ordinateur d'un utilisateur, on évitera de stocker des informations sensibles.
expire	La date d'expiration du cookie sous forme d'un timestamp UNIX (nombre de secondes écoulées depuis le 1er janvier 1970). Si aucune valeur n'est passée en argument, le cookie expirera à la fin de la session (lorsque le navigateur sera fermé).
path	Le chemin sur le serveur sur lequel le cookie sera disponible. Si la valeur est '/', le cookie sera disponible sur l'ensemble du domaine. Si la valeur est '/cours/', le cookie ne sera disponible que dans le répertoire (le dossier) /cours/ (et dans tous les sous-répertoires qu'il contient).
domain	Indique le domaine ou le sous domaine pour lequel le cookie est disponible.
secure	Indique si le cookie doit uniquement être transmis à travers une connexion sécurisée HTTPS depuis le client. Si la valeur passée est <code>true</code> , le cookie ne sera envoyé que si la connexion est sécurisée.
httponly	Indique si le cookie ne doit être accessible que par le protocole HTTP. Pour que le cookie ne soit accessible que par le protocole http, on indiquera la valeur <code>true</code> . Cela permet d'interdire l'accès au cookie aux langages de scripts comme le JavaScript par exemple, pour se protéger potentiellement d'une attaque de type XSS.

Exemple :

```
<?php
    setcookie('user_id', '1234', time()+3600, null, null, false, true);
?>
<!DOCTYPE html>
<html lang="fr">

<head>
```

2.3 Récupérer la valeur d'un cookie

Pour récupérer la valeur d'un cookie, nous allons utiliser la variables superglobale `$_COOKIE`.

Cette superglobale est un tableau associatif qui utilise les noms des cookies en clefs et associe leurs valeurs en valeurs du tableau.

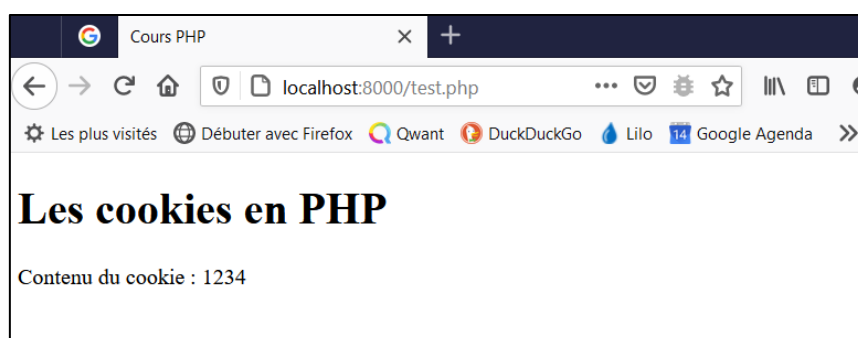
On va donc pouvoir accéder à la valeur d'un cookie en particulier en renseignant le nom du cookie en clef de ce tableau.

```
<?php
    setcookie('user_id', '1234', time()+3600, null, null, false, true);
?>
<!DOCTYPE html>
<html lang="fr">

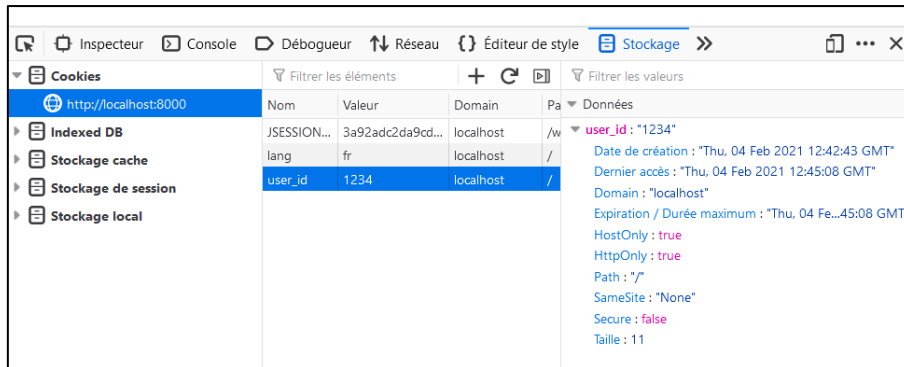
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Cours PHP</title>
</head>

<body>
    <h1>Les cookies en PHP</h1>
    <p>Contenu du cookie : <?= $_COOKIE['user_id'] ?></p>
</body>

</html>
```



Il est possible d'examiner les cookies avec les outils développeurs des navigateurs :



2.4 Modifier ou supprimer un cookie

Pour modifier la valeur d'un cookie, nous allons appeler à nouveau la fonction `setcookie()` en lui passant le nom du cookie dont on souhaite changer la valeur et changer l'argument de type valeur passé à la fonction avec la nouvelle valeur souhaitée.

Pour supprimer un cookie, nous allons encore appeler `setcookie()` en lui passant le nom du cookie qu'on souhaite supprimer et allons cette fois-ci définir une date d'expiration se situant dans le passé pour le cookie en question.

```
<?php
// définition du cookie
setcookie('user_id', '1234', time()+3600, null, null, false, true);

// modification du cookie
setcookie('user_id', '5678', time()+3600, null, null, false, true);

// suppression du cookie
setcookie('user_id', '', time()-3600, null, null, false, true);
?>
```

Vidéo [Grafikart sur les cookies](#).

3 Définir et utiliser les sessions en PHP

3.1 Présentation des sessions en PHP

Une session en PHP correspond à une façon de stocker des données différentes pour chaque utilisateur en utilisant un identifiant de session unique.

Les identifiants de session vont généralement être envoyés au navigateur via des cookies de session et vont être utilisés pour récupérer les données existantes de la session.

Un des grands intérêts des sessions est qu'on va pouvoir conserver des informations pour un utilisateur lorsqu'il navigue d'une page à une autre. De plus, les informations de session ne vont cette fois-ci pas être stockées sur les ordinateurs de vos visiteurs à la différence des cookies mais plutôt côté serveur ce qui fait que les sessions vont pouvoir être beaucoup plus sûres que les cookies.

Notez toutefois que le but des sessions n'est pas de conserver des informations indéfiniment mais simplement durant une « session ». Une session démarre dès que la fonction `session_start()` est appelée et se termine en général dès que la fenêtre courante du navigateur est fermée (à moins qu'on appelle une fonction pour terminer la session de manière anticipée ou qu'un cookie de session avec une durée de vie plus longue ait été défini).

La superglobale `$_SESSION` est un tableau associatif qui va contenir toutes les données de session une fois la session démarrée.

3.2 Démarrer une session en PHP

Pour démarrer une session en PHP, on va utiliser la fonction `session_start()`. Cette fonction va se charger de vérifier si une session a déjà été démarrée en recherchant la présence d'un identifiant de session et, si ce n'est pas le cas, va démarrer une nouvelle session et générer un identifiant de session unique pour un utilisateur.

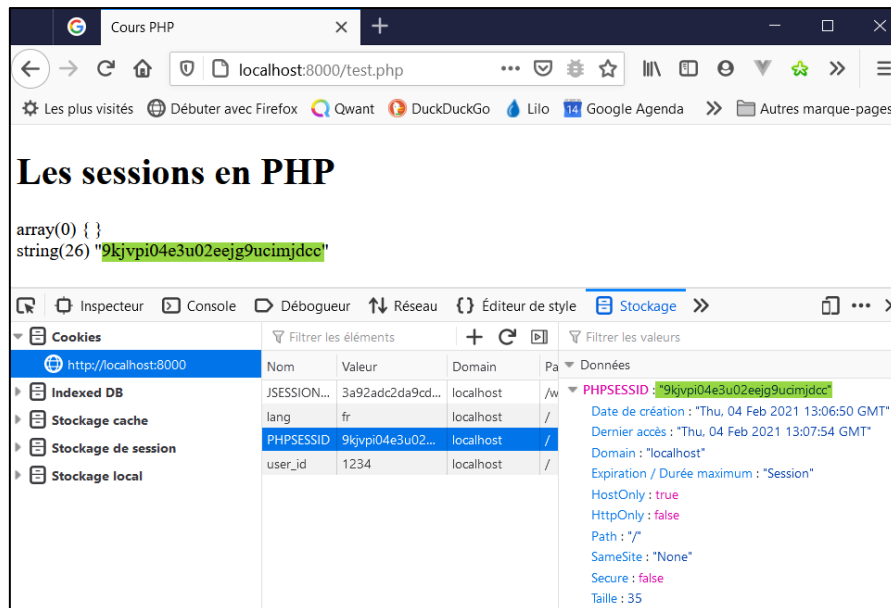
Il va falloir appeler `session_start()` avant toute autre opération dans nos pages, c'est-à-dire au début de celles-ci de la même façon qu'on a déjà pu le faire avec la fonction `setcookie()`.

Par ailleurs, notez qu'il va falloir appeler `session_start()` dans chaque page où on souhaite pouvoir accéder aux variables de session.

```
<?php
    session_start();
?>
<!DOCTYPE html>
<html lang="fr">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Cours PHP</title>
</head>

<body>
    <h1>Les sessions en PHP</h1>
    <?php
        var_dump($_SESSION);
        echo '<br>';
        var_dump(session_id());
    ?>
</body>
</html>
```



Pour le moment notre session est vide mais nous pouvons la repérer grâce à `session_id()`.

3.3 Définir et récupérer des variables de session

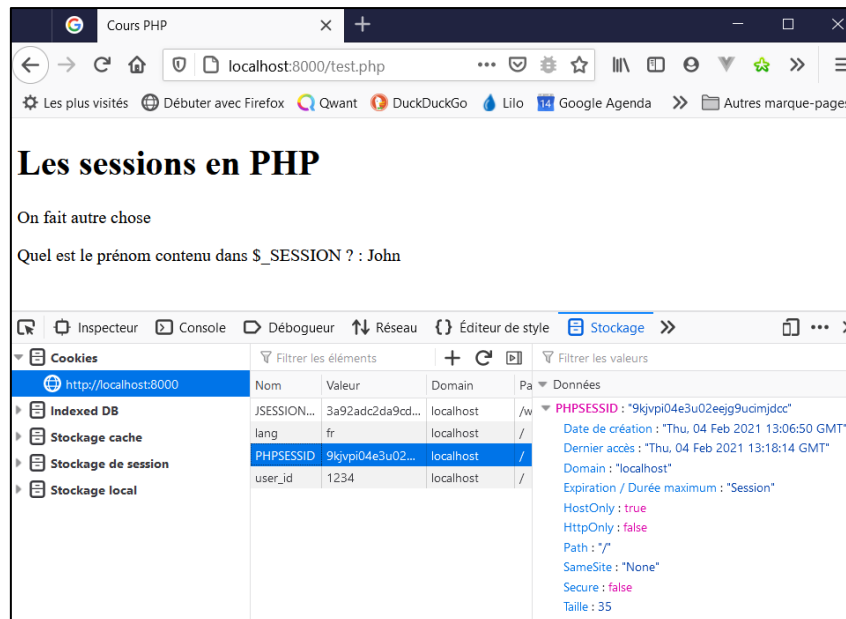
Pour définir et récupérer les valeurs des variables de session, nous allons pouvoir utiliser `$_SESSION` qui est un tableau associatif.

Examinons le code suivant et son retour :

```
<?php
    session_start();
?>
<!DOCTYPE html>
<html lang="fr">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Cours PHP</title>
</head>

<body>
    <h1>Les sessions en PHP</h1>
    <?php
        $_SESSION['prenom'] = 'John';
    ?>
    <p>On fait autre chose</p>
    <p>Quel est le prénom contenu dans $_SESSION ? : <?=
$_SESSION['prenom'] ?></p>
</body>
</html>
```



Deux choses sont à remarquer :

- Nous pouvons « alimenter » la session n'importe quand et de n'importe où.
- Le cookie **PHPSESSID** ne contient que l'id de la session afin de permettre au serveur de le reconnaître. Le contenu reste sur le serveur.

3.4 Terminer une session et détruire les variables de session

Une session PHP se termine généralement automatiquement lorsqu'un utilisateur ferme la fenêtre de son navigateur.

La fonction **session_destroy()** va supprimer le fichier de session dans lequel sont stockées toutes les informations de session. Cependant, cette fonction ne détruit pas les variables globales associées à la session (c'est-à-dire le contenu du tableau **\$_SESSION**) ni le cookie de session.

Pour détruire totalement une session, il va également falloir supprimer l'identifiant de session. Généralement, cet identifiant est contenu dans le cookie **PHPSESSID** qu'on pourra effacer en utilisant **setcookie()** en définissant une date d'expiration passée pour le cookie.

Il va cependant être très rare d'avoir besoin de détruire les données associées à une session et donc d'appeler **session_destroy()**. On préférera généralement modifier le tableau **\$_SESSION** manuellement pour supprimer des données en particulier.

Notez qu'on va également pouvoir utiliser la fonction **session_unset()** (sans lui passer d'argument) pour détruire toutes les variables de la session courante. Cette fonction va également nous permettre de détruire une variable de session en particulier en lui passant sa valeur de la manière suivante : **unset(\$_SESSION['nom-de-la-variable-de-session-a-detruire'])**.

Documentation PHP sur les [sessions](#).

Vidéo [Grafikart sur les sessions](#).